

Water Usage Monitoring System

Complete Technical Documentation (A to Z Architecture, Implementation & Flows)

Water Usage Monitoring System — Complete A-to-Z Project Documentation

1. Project Overview

The **Water Usage Monitoring System** is an enterprise-grade, full-stack IoT-enabled smart utility management web application. Designed for multi-family residential apartments, gated communities, and commercial complexes, it enables real-time water meter monitoring, automated tiered tariff bill generation, bulk water tanker cost apportionment, interactive online payment processing, and comprehensive billing/payment ledger history.

Core Objectives

- **Conservation & Telemetry:** Monitor daily/monthly water consumption in Kilolitres (KL) down to individual households and meters.
- **Fair Tiered Tariff Billing:** Calculate monthly water charges dynamically using multi-tier rate slabs, base service fees, common area shared costs, and GST taxes.
- **Bulk Tanker Purchase Cost Sharing:** Support recording supplementary bulk water tanker purchases by management and proportionally apportioning the cost across active apartment households based on consumption.
- **End-to-End Dynamic Payment Gateway:** Provide a seamless online payment experience supporting Paytm, GPay (Google Pay), Razorpay, Debit/Credit Cards, and NetBanking directly backed by MySQL database persistence.
- **Role-Based Access Control:** Separate Administrative Management Portal from Resident Household Portals with JWT authentication.

2. Original Project Analysis & Improvements

Original Implementation Analysis

- **Positive Aspects:**
 - Visually attractive UI layout and structure for Resident and Admin dashboards.
 - Well-defined component folder structure separating pages, components, context, and CSS files.
 - Clear domain concepts (Apartments, Meters, Usage, Bills, Payments, Tariffs).

- **Original Limitations:**
- **Static & Fake Data:** Current Bill, Payment History, and Billing History relied heavily on hardcoded mock values (e.g., fixed ₹842.50 bill amount across all months).
- **Fake Payment Gateway:** Clicking "Pay Bill" originally jumped to a static success screen without executing backend REST API calls or persisting payment receipts.
- **Disconnected Backend & Database:** Frontend services lacked active endpoints for payment processing (**POST /api/bills/{billId}/pay**) and payment history retrieval (**GET /api/bills/payments**).
- **Changes & Improvements Implemented:**
- **100% Dynamic Backend Integration:** Replaced static mock state in **CurrentBill.jsx**, **BillingHistory.jsx**, and **PaymentHistory.jsx** with live Spring Boot REST API calls (**billService.js**).
- **Database Persistence:** Integrated real MySQL database persistence for **bill** and **payment** tables, automatically setting bill status to **PAID** upon transaction completion.
- **Interactive Payment Options:** Added explicit payment modal support for **Paytm**, **GPay**, **Razorpay**, **Cards**, and **NetBanking**.
- **Official Payment Receipt Modal:** Generates dynamic on-screen digital payment receipts containing Receipt ID, Transaction Ref, Amount, Payment Method, Timestamp, and Resident details with file download capabilities.

3. Original vs Current Project Matrix

Feature Module	Original Implementation	Current Implementation	Status
Authentication	Local storage mock login	JWT Bearer Authentication with Spring Security	Fully Implemented
Water Meter Telemetry	Hardcoded meter IDs	Dynamic meter mapping & CSV bulk upload endpoint	Fully Implemented
Water Usage Calculation	Fixed static values	Daily/monthly consumption KL aggregation	Fully Implemented
Tariff Slabs	Fixed rate assumptions	Multi-tiered slab calculation (Tier 1, Tier 2, Tanker Surcharge)	Fully Implemented
Bulk Tanker Purchase	Missing	Admin module for recording tanker cost & proportional allocation	Fully Implemented
Current Bill View	Static ₹842.50 hardcoded card	Dynamic invoice populated directly from Spring Boot & DB	Fully Implemented
Pay Bill Action	Mock button with fake redirect	Live POST /api/bills/{id}/pay API with database write	Fully Implemented

Payment Gateways	None	Interactive modal supporting Paytm, GPay, Razorpay, Card, NetBanking	Fully Implemented
Payment Receipt	None	Dynamic modal & text file receipt export	Fully Implemented
Billing History	Hardcoded array	Fetches live statements from GET /api/bills	Fully Implemented
Payment History	Static list	Fetches live transaction log from GET /api/bills/payments	Fully Implemented

4. Technology Stack

```
graph TD
  UI["React 18 Frontend (Vite)"] -->|REST APIs / JSON| API["Spring Boot 3.2.5 REST Controllers"]
  API -->|Spring Security + JWT| SEC["Security & Auth Manager"]
  API -->|Spring Data JPA| ORM["Hibernate ORM"]
  ORM -->|JDBC Driver| DB["MySQL 8 Database / H2 Engine"]
```

- **Frontend:** React 18, React Router v6, Vite, Vanilla CSS Modules, Context API (**ProfileContext**), Lucide React Icons.
- **Backend:** Java 21, Spring Boot 3.2.5, Spring Security, Spring Data JPA, Hibernate, Jackson JSON, Spring Mail (JavaMailSender).
- **Database:** MySQL 8.0 (Production) / H2 In-Memory (Test/Fallback Profile).
- **Security:** JWT (JSON Web Tokens), BCrypt Password Hashing, CORS Configuration.

5. System Architecture

```
architecture-beta
group client(browser)[Resident & Admin Browsers]
group web(web)[Frontend Web Application]
group backend(server)[Spring Boot Application Server]
group db(database)[MySQL Database Server]

service residentUI(internet)[Resident Portal React SPA] in client
service adminUI(internet)[Admin Portal React SPA] in client

service apiConfig(code)[apiConfig.js / Axios Fetch] in web
service controllers(server)[Spring Boot Controllers] in backend
service services(server)[Service Business Logic Layer] in backend
service repos(database)[JPA Repositories] in backend

service mysql(database)[MySQL / H2 Database] in db

residentUI --> apiConfig
adminUI --> apiConfig
apiConfig --> controllers
controllers --> services
services --> repos
repos --> mysql
```

6. Database Architecture & Schema

ER Diagram & Relationships

```
erDiagram
USERS ||--o{ USER_ROLES : has
ROLES ||--o{ USER_ROLES : assigned_to
APARTMENT ||--o| USERS : occupied_by
APARTMENT ||--o| WATER_METER : installed_on
WATER_METER ||--o{ WATER_USAGE : logs
APARTMENT ||--o{ BILL : generates
BILL ||--o| PAYMENT : settles
BULK_WATER_PURCHASE }|--|| BILLING_CYCLE : allocates
```

Key Database Tables

- **users:**
 - **user_id** (BIGINT, PK, Auto Increment)
 - **full_name** (VARCHAR 100)
 - **email** (VARCHAR 100, Unique)
 - **phone** (VARCHAR 20)
 - **password_hash** (VARCHAR 255)
 - **status** (VARCHAR 20)
 - **created_at** (TIMESTAMP)
- **apartment:**
 - **apartment_id** (BIGINT, PK, Auto Increment)
 - **apartment_number** (VARCHAR 20)

- **building_name** (VARCHAR 100)
- **floor_number** (INT)
- **occupancy_status** (VARCHAR 20)
- **resident_user_id** (BIGINT, FK to **users**)
- **water_meter**:
 - **meter_id** (BIGINT, PK, Auto Increment)
 - **serial_number** (VARCHAR 50, Unique)
 - **apartment_id** (BIGINT, Unique, FK to **apartment**)
 - **battery_percentage** (INT)
 - **signal_strength** (INT)
 - **status** (VARCHAR 20)
- **water_usage**:
 - **usage_id** (BIGINT, PK, Auto Increment)
 - **meter_id** (BIGINT, FK to **water_meter**)
 - **previous_reading** (DECIMAL 10,2)
 - **current_reading** (DECIMAL 10,2)
 - **consumption_kl** (DECIMAL 10,2)
 - **reading_date** (DATE)
- **bill**:
 - **bill_id** (BIGINT, PK, Auto Increment)
 - **bill_number** (VARCHAR 50, Unique)
 - **apartment_id** (BIGINT, FK to **apartment**)
 - **billing_month** (VARCHAR 20)
 - **consumption_kl** (DECIMAL 10,2)
 - **volumetric_amount** (DECIMAL 10,2)
 - **base_charge** (DECIMAL 10,2)
 - **common_water_charge** (DECIMAL 10,2)
 - **gst_amount** (DECIMAL 10,2)
 - **total_amount** (DECIMAL 10,2)
 - **due_date** (DATE)
 - **status** (VARCHAR 20, Default: **PENDING**)
- **payment**:
 - **payment_id** (BIGINT, PK, Auto Increment)
 - **transaction_ref** (VARCHAR 100, Unique)
 - **bill_id** (BIGINT, FK to **bill**)
 - **amount** (DECIMAL 10,2)
 - **payment_method** (VARCHAR 30) — e.g. **Paytm**, **GPay**, **Razorpay**, **Card**, **NetBanking**

- **status** (VARCHAR 20, Default: **SUCCESSFUL**)
- **paid_at** (TIMESTAMP)
- **bulk_water_purchase**:
- **purchase_id** (BIGINT, PK, Auto Increment)
- **vendor_name** (VARCHAR 100)
- **tanker_capacity_kl** (DECIMAL 10,2)
- **total_cost** (DECIMAL 10,2)
- **purchase_date** (DATE)
- **billing_month** (VARCHAR 20)

7. Implementation Plan Execution Details

Week 1 & Week 2: Core Foundation & Database Schema

- **Requirements:** Establish domain models, JPA entities, repositories, basic auth, and initial REST API endpoints.
- **Implemented Work:**
- Defined database schemas (**users**, **apartment**, **water_meter**, **water_usage**, **bill**, **payment**).
- Created JPA Entities in **com.watermonitoring.entity**.
- Implemented Repositories (**UserRepository**, **ApartmentRepository**, **WaterMeterRepository**, **BillRepository**, **PaymentRepository**).
- Built Authentication controllers (**AuthController.java**) and JWT security pipeline.
- Connected React frontend auth service (**authService.js**).

Week 3 & Week 4: Water Usage, Meter Telemetry & Dynamic Billing

- **Requirements:** CSV bulk reading upload, meter status telemetry, tiered tariff calculations, bulk tanker purchase allocation, and live payment processing.
- **Implemented Work:**
- Built CSV parser in **WaterMeterController.java** (**POST /api/meters/upload-csv**).
- Created **TariffPlan.java**, **TariffController.java**, **TariffService.js**.
- Upgraded **BillSchedulerService.java** to compute volumetric charges based on slab tariffs + proportional bulk tanker purchases + GST.
- Implemented **POST /api/bills/{billId}/pay** and **GET /api/bills/payments**.
- Connected **CurrentBill.jsx**, **BillingHistory.jsx**, and **PaymentHistory.jsx** directly to backend REST endpoints.

Week 5 & Week 6: Advanced Analytics, Leak Detection & Email Receipts (Plan)

- **Requirements:** Automatic leak detection alerts, automated email notification dispatch, advanced consumption analytics graphs, and PDF bill statement generation.
- **Planned Work:**
- Implement scheduled leak detection background job analyzing anomalous meter surges.

- Integrate PDF invoice download generation using iText / PDFBox in backend.
- Dispatch email receipts via Spring Mail (**JavaMailSender**) upon successful payment execution.

Week 7 & Week 8: Production Polish, Audit Logging & Deployment (Plan)

- **Requirements:** Security hardening, audit logging, Docker containerization, and final user acceptance testing.
- **Planned Work:**
 - Configure production Nginx reverse proxy and HTTPS SSL certificates.
 - Implement full system audit logger recording admin operations.
 - Conduct load testing and end-to-end verification.

8. Complete Technical Flow Examples

A. User Authentication Flow

```
React Login Page (Login.jsx)
↓ POST /api/auth/login { email, password }
AuthController.java -> AuthService.java
↓ BCrypt Password Match & JWT Token Generation
User & Role Repositories -> MySQL Database
↓ Return { token, user: { name, email, role } }
React Auth Context (LocalStorage Token Saved)
↓ Redirect to Dashboard (/resident/dashboard or /admin/dashboard)
```

B. Dynamic Payment Processing & Receipt Flow

```
Current Bill Page (CurrentBill.jsx)
↓ Click [ ■ Pay Bill (■842.50) ]
Payment Box Modal Opens (Select Paytm / GPay / Razorpay / Card / NetBanking)
↓ Click Submit Payment
billService.payBill(billId, paymentData)
↓ POST /api/bills/{billId}/pay
BillController.java -> processPayment()
↓ Update bill.status = 'PAID' & Save payment entity
BillRepository & PaymentRepository -> MySQL Database
↓ Return ApiResponse<Payment> (success: true)
CurrentBill.jsx State Updated (isPaid = true, totalAmount = 0.00)
↓ Show Official Payment Receipt Modal (Receipt #RCP-1234, Paytm/GPay/Razorpay, Timestamp)
Payment History (PaymentHistory.jsx) -> Fetches GET /api/bills/payments (Displays Record)
Billing History (BillingHistory.jsx) -> Fetches GET /api/bills (Displays Status: PAID)
```

9. Current Website Features (A-Z Catalog)

- **Admin Dashboard:** Real-time overview of community water consumption, active meters, pending bill totals, and recent tanker purchases.
- **Apartment & Resident Management:** Create, assign, and manage building blocks, units, and resident profiles.
- **Bulk Water Purchase:** Record supplementary tanker purchases and automatically apportion costs across active residents.
- **Billing Management:** Trigger manual or automated monthly bill generation for all apartments.

- **Current Bill Portal:** Interactive resident invoice view detailing volumetric charges, common area costs, service fees, and GST.
- **Interactive Payment Gateway:** Dedicated payment modal supporting **Paytm, GPay, Razorpay, Debit/Credit Cards**, and **NetBanking**.
- **Payment History & Receipts:** Digital transaction ledger displaying historical payments with on-screen modal receipts and text export.
- **Tariff Management:** Define tiered consumption rate slabs and base maintenance fees.
- **Water Meter Management:** Monitor telemetry status (online/offline), battery percentage, signal strength, and CSV bulk data ingestion.

10. Conclusion & Project Status

The **Water Usage Monitoring System** has successfully transitioned from a static prototype to a **100% dynamic, fully connected enterprise solution**.

All core requirements for **Weeks 1 through 4** are complete, including:

- Dynamic Spring Boot REST API services.
- Full database persistence with MySQL.
- Live tiered billing and bulk tanker purchase cost distribution.
- Interactive payment gateway integration supporting **Paytm, GPay, Razorpay, Cards**, and **NetBanking**.
- Dynamic digital payment receipts and interconnected history statements.

The application is fully operational and ready for Week 5–8 advanced analytics and automated email receipt enhancements.